

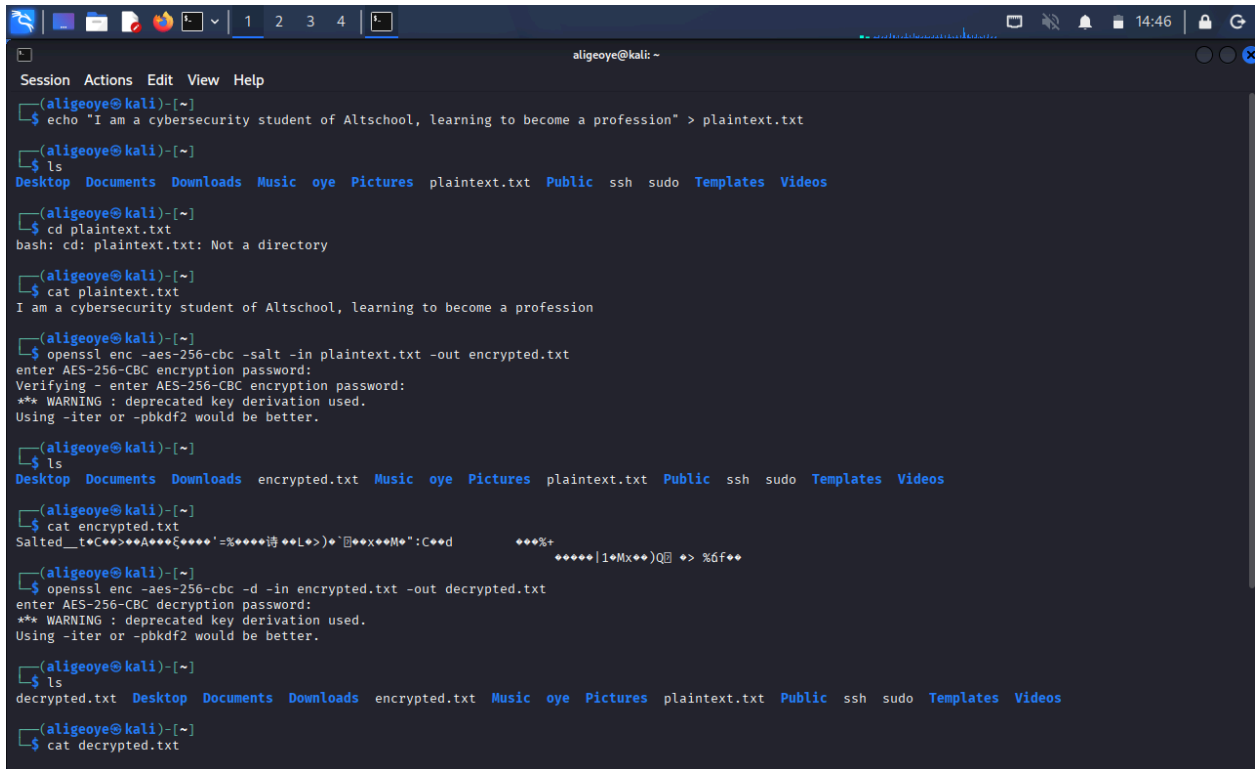
Practical reporting and demonstration of symmetry and asymmetry encryption using Kali

1. SYMMETRY ENCRYPTION

First I created a file name "plaintext.txt"

I encrypted with the following command "openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.txt"

It was saved as "encrypted.txt"



```
aligeoye@kali: ~  
Session Actions Edit View Help  
(aligeoye@kali)-[~]  
$ echo "I am a cybersecurity student of Altschool, learning to become a profession" > plaintext.txt  
(aligeoye@kali)-[~]  
$ ls  
Desktop Documents Downloads Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
(aligeoye@kali)-[~]  
$ cd plaintext.txt  
bash: cd: plaintext.txt: Not a directory  
(aligeoye@kali)-[~]  
$ cat plaintext.txt  
I am a cybersecurity student of Altschool, learning to become a profession  
(aligeoye@kali)-[~]  
$ openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.txt  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.  
(aligeoye@kali)-[~]  
$ ls  
Desktop Documents Downloads encrypted.txt Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
(aligeoye@kali)-[~]  
$ cat encrypted.txt  
Salted__tC**>+A***{****'=%*****i$ **L*)*`[***x**Me":C*d      ***%+  
*****|1Mx***)Q$ **> %dF**  
(aligeoye@kali)-[~]  
$ openssl enc -aes-256-cbc -d -in encrypted.txt -out decrypted.txt  
enter AES-256-CBC decryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.  
(aligeoye@kali)-[~]  
$ ls  
decrypted.txt Desktop Documents Downloads encrypted.txt Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
(aligeoye@kali)-[~]  
$ cat decrypted.txt
```

I decrypted the file with the following command "openssl enc -aes-256-cbc -d -in encrypted.txt -out decrypted.txt"

It was saved as "decrypted.txt"

```
aligeoye@kali: ~  
Session Actions Edit View Help  
aligeoye@kali-[-~]  
$ ls  
Desktop Documents Downloads Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
aligeoye@kali-[-~]  
$ cd plaintext.txt  
bash: cd: plaintext.txt: Not a directory  
aligeoye@kali-[-~]  
$ cat plaintext.txt  
I am a cybersecurity student of Altschool, learning to become a profession  
aligeoye@kali-[-~]  
$ openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.txt  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.  
aligeoye@kali-[-~]  
$ ls  
Desktop Documents Downloads encrypted.txt Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
aligeoye@kali-[-~]  
$ cat encrypted.txt  
Salted__tC++A++++'=%*****L>)*`[]x++M*":C+d      ***%+  
*****[1Mx++)Q[] >> %6f++  
aligeoye@kali-[-~]  
$ openssl enc -aes-256-cbc -d -in encrypted.txt -out decrypted.txt  
enter AES-256-CBC decryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.  
aligeoye@kali-[-~]  
$ ls  
decrypted.txt Desktop Documents Downloads encrypted.txt Music oye Pictures plaintext.txt Public ssh sudo Templates Videos  
aligeoye@kali-[-~]  
$ cat decrypted.txt  
I am a cybersecurity student of Altschool, learning to become a profession  
aligeoye@kali-[-~]
```

When I cat the file it was the exact document I encrypted earlier.

2. ASYMMETRIC ENCRYPTION

Generate a private key using this command “openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048”

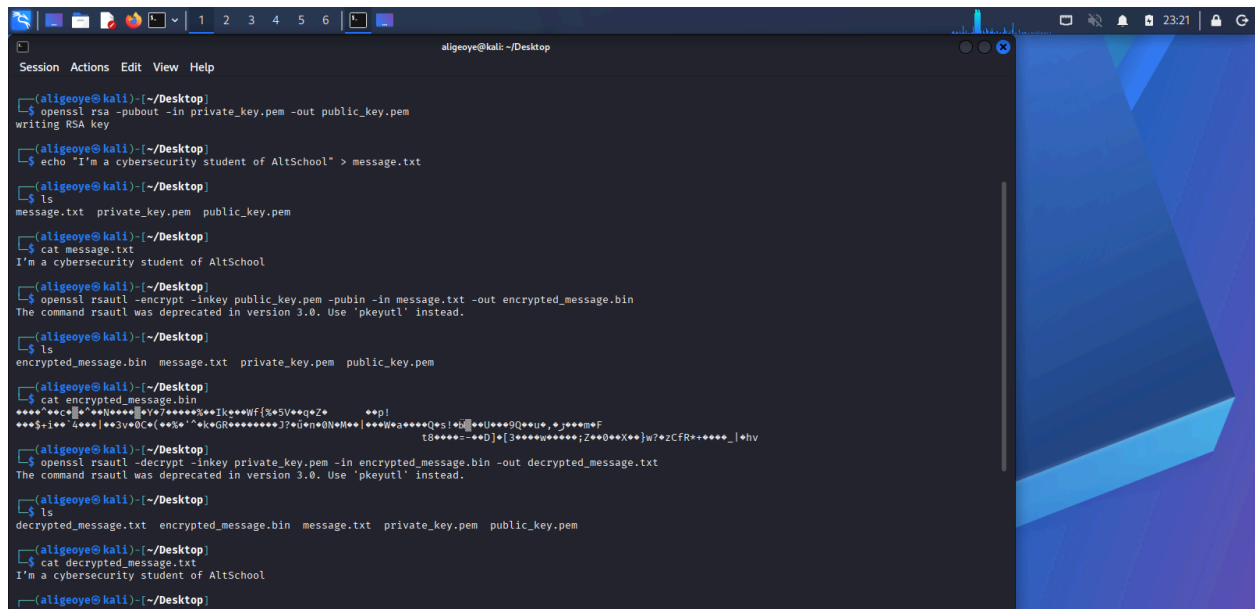
I created a public key from the private key using this command “openssl rsa -pubout -in private_key.pem -out public_key.pem”

I created a file with the following command “echo "I'm a cybersecurity student of AltSchool" > message.txt”

I encrypted the file using the public key with this command “openssl rsautl -encrypt -inkey public_key.pem -pubin -in message.txt -out encrypted_message.bin”

[illegible]

Decrypted file showing the exact document



```
aligeoye@kali: ~/Desktop
Session Actions Edit View Help

aligeoye@kali:~/Desktop
$ openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key

aligeoye@kali:~/Desktop
$ echo "I'm a cybersecurity student of AltSchool" > message.txt

aligeoye@kali:~/Desktop
$ ls
message.txt  private_key.pem  public_key.pem

aligeoye@kali:~/Desktop
$ cat message.txt
I'm a cybersecurity student of AltSchool

aligeoye@kali:~/Desktop
$ openssl rsautl -encrypt -inkey public_key.pem -pubin -in message.txt -out encrypted_message.bin
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

aligeoye@kali:~/Desktop
$ ls
encrypted_message.bin  message.txt  private_key.pem  public_key.pem

aligeoye@kali:~/Desktop
$ cat encrypted_message.bin
****"acall"~*o*****o?*****kcoowf{5sVee+qe+Ze      **p!
***$:-!ee  A***3v*0C*(**%*^k*GR*****})*un*0N*H*ee|***j******Q*es|*i|*eej***9Q*ee;*.+*****F
                                     t8*****--**D|*{3*****u*****;Z*0**X**}w?*zCfR+*****_l*hv

aligeoye@kali:~/Desktop
$ openssl rsautl -decrypt -inkey private_key.pem -in encrypted_message.bin -out decrypted_message.txt
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

aligeoye@kali:~/Desktop
$ ls
decrypted_message.txt  encrypted_message.bin  message.txt  private_key.pem  public_key.pem

aligeoye@kali:~/Desktop
$ cat decrypted_message.txt
I'm a cybersecurity student of AltSchool

aligeoye@kali:~/Desktop
```

3. WEB SERVER USING APACHE

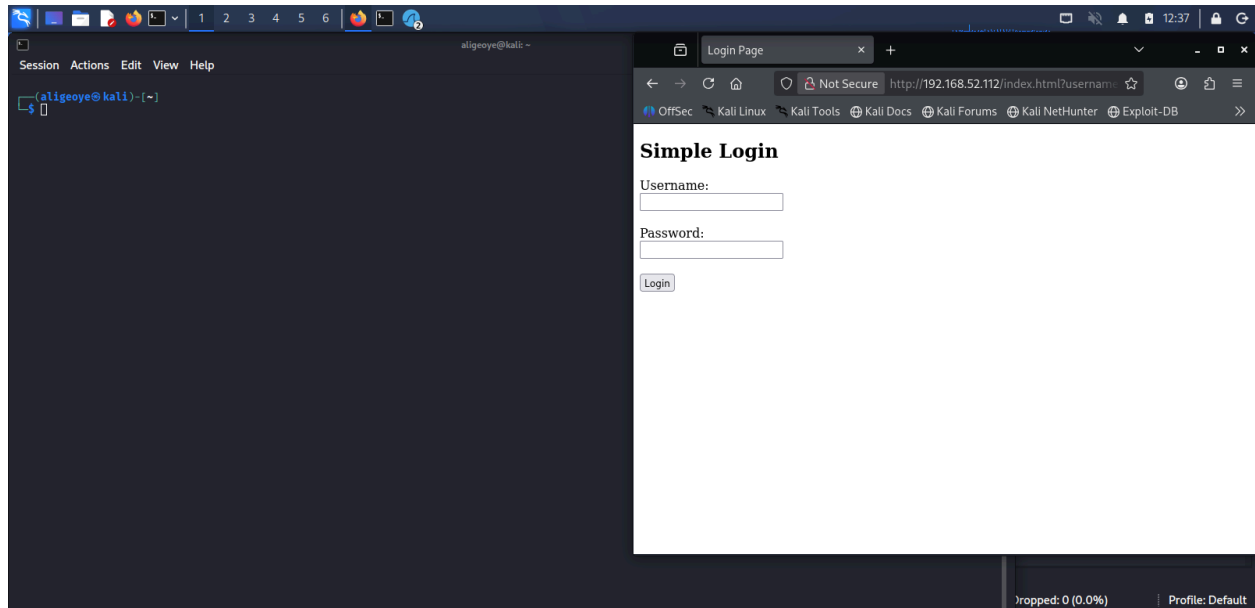
I configured a simple web page in Kali Linux using apache with the following command

```
sudo apt install apache2
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2
sudo systemctl restart apache2
```

```
cd /var/www/html
ls
nano index.html
```

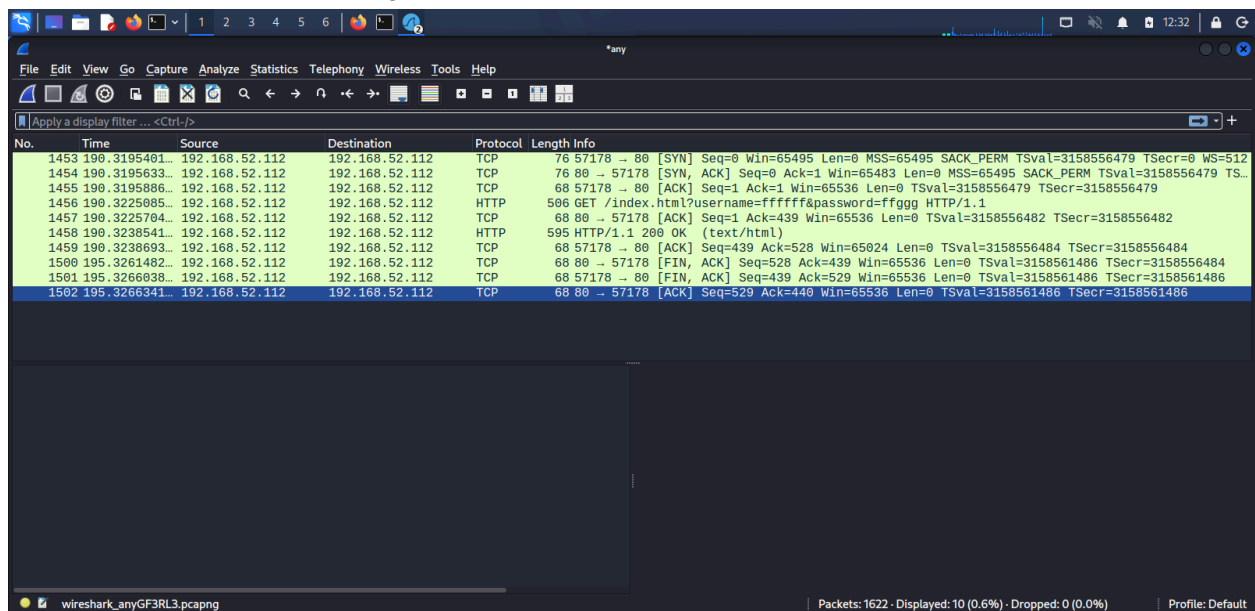
I wrote my code in the nano page and saved.

After saving I login through a web browser and it was access through http

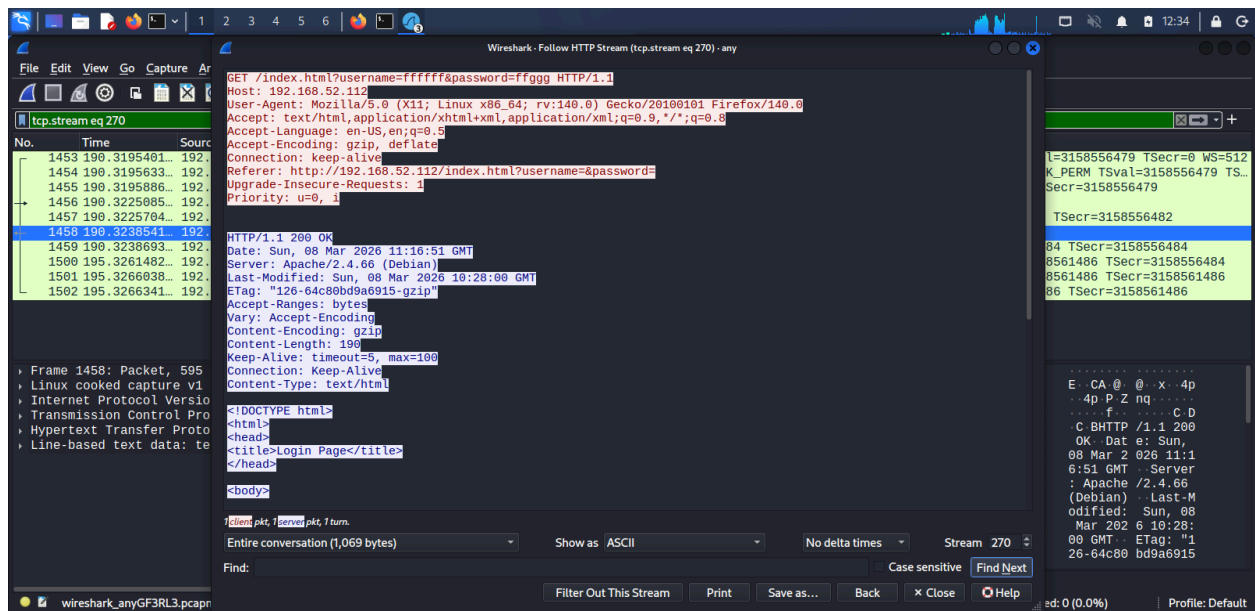


Webpage loaded over HTTP

I captured some packets using wireshark.



Http packets was shown to be unencrypted.



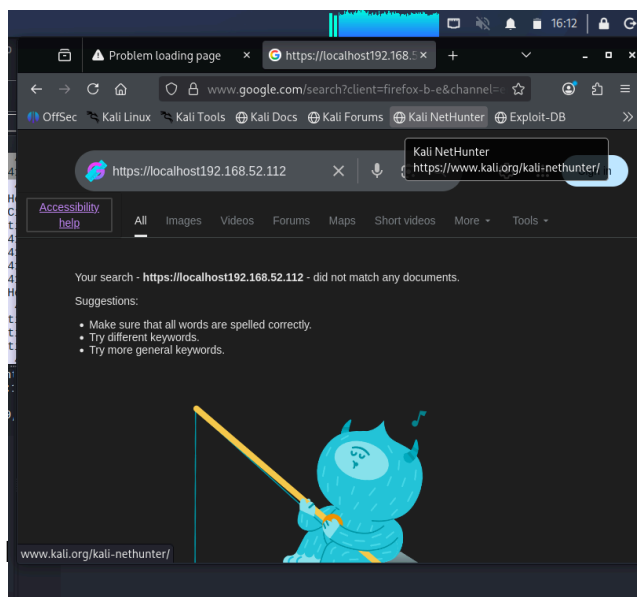
Packets was readable

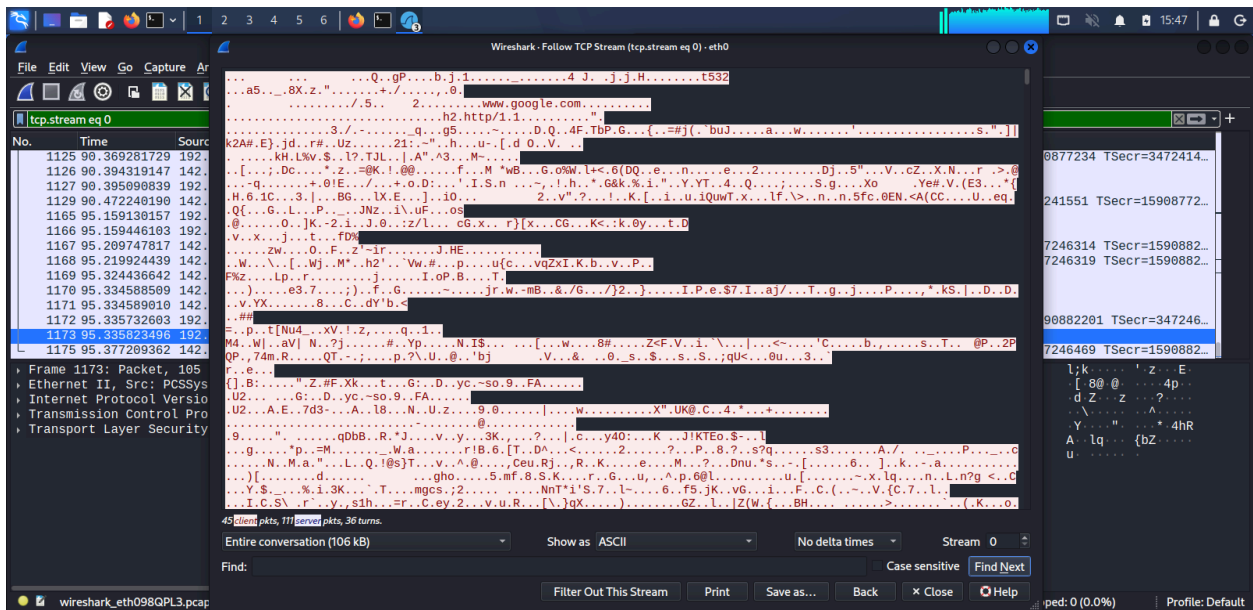
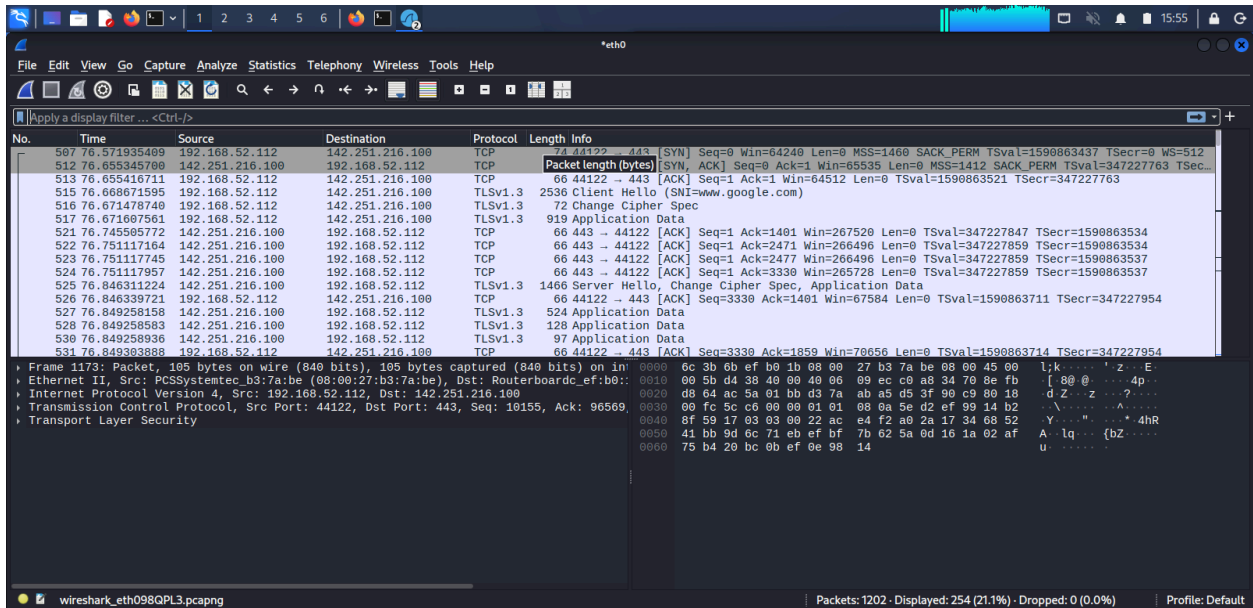
4. HTTPS CONFIGURATION WITH SELF-SIGNED CERTIFICATE

Generate a private key using this command “sudo openssl genrsa -out /etc/ssl/private/myserver.key 2048”

I created a self-signed certificate

I configured a web server for https





Wireshark showing encrypted application data